

NavLink v2

Protocol Analysis & a MAVLink-Class Scalable Redesign

Vayu flight stack · `src/comm, software/src/protocol, tools/autotune`

June 2026

Abstract

An analysis of the as-built **NavLink v1** link protocol — its wire format, message catalog, and framing — followed by a detailed redesign, **NavLink v2**, that scales like MAVLink while keeping NavLink’s compactness. Covers v1’s structural limitations (a 4-bit message-ID ceiling, no version-safe decoding, no sequence numbers, no addressing, no acknowledged delivery), the v2 frame format with addressing/sequencing/`CRC_EXTRA`, field-truncation for forward and backward compatibility, the command/parameter/mission/file service layers, toggleable AEAD security (per-frame signing and/or encryption), a single-source-of-truth codegen strategy, a phased v1 v2 coexistence migration, and the new packets the vehicle needs next (GPS, battery, position, ESC telemetry, and more).

Contents

1. Executive Summary	2
2. NavLink v1 — As-Built	2
2.1 Wire format	2
2.2 Message catalog (v1, as-built)	3
2.3 Framing & dispatch	4
2.4 Bandwidth (current load)	4
3. Why v1 Doesn’t Scale — Limitations	4
L1 — 4-bit message-ID space (hard ceiling)	4
L2 — No wire-format integrity across versions (<code>CRC_EXTRA</code> gap)	4
L3 — No sequence numbers → packet loss is invisible	4
L4 — No real addressing / routing	4
L5 — No acknowledged delivery	5
L6 — Two parallel, inconsistent ID spaces	5
L7 — No forward/backward field compatibility	5
L8 — Three hand-maintained codecs, no source of truth	5
L9 — Weak resync, small RX buffers	5
L10 — Miscellaneous	5
4. Design Goals for v2	5
5. NavLink v2 — Frame Format	6
5.1 Sizes & limits	7
6. <code>CRC_EXTRA</code> — Version-Safe Decoding	7
7. Field Encoding & Payload Truncation (forward/backward compat)	8
7.1 Time fields	8
7.2 Numeric types & fixed-point fields	8
8. Service Layers (the “microservices”)	9
8.1 Command service — typed commands, no float box	9
8.2 Parameter service — typed values, transactional dumps	9
8.3 Mission service — waypoints / geofence / rally	10
8.4 Bulk transfer (logs, large blobs) — fixes the 255-byte cap	10

8.5 Security — authentication & encryption (toggleable)	10
8.6 Time synchronization — the epoch behind every payload timestamp	12
9. Migration Plan (v1 v2 coexistence)	12
Phase 0 — Catalog & codegen (no wire change)	12
Phase 1 — v2 framing library, behind a build flag	14
Phase 2 — Dual-emit telemetry	14
Phase 3 — Command/param/mission services	14
Phase 4 — Retire v1	14
Code touch-points (for sizing)	14
10. Proposed Message Catalog (v2)	14
10.1 Core / system (0–255) — port of today	15
10.2 Sensors / state (1024–2047) — port + extend	16
10.3 Navigation / position (2048–3071) — new, needed soon	17
10.4 Power / propulsion (3072–4095) — new, needed soon	17
10.5 Services (12288–16383)	17
10.6 Payloads / peripherals (4096–8191)	17
10.7 Commands (8192–12287) — typed, first-class	18
11. Risks & Trade-offs	18
12. Summary Table — v1 vs v2	18
13. Improving on MAVLink	19

1. Executive Summary

NavLink v1 is a compact, CRC-protected, byte-framed link that does exactly one job well: stream telemetry down and commands up over a single UART between one flight controller and one GCS. It is easy to read and cheap on an STM32F4.

It does *not* scale. The wire format hard-caps the protocol at **16 message types** (a 4-bit field), has **no sequence numbers** (so packet loss is invisible), **no addressing** beyond a single informational `device_id`, **no wire-format integrity across versions** (a sender and receiver that disagree about a message’s layout will silently mis-decode it), **no acknowledged delivery**, and requires editing **three** codebases by hand to add any message. The `SYSTEM_STATUS` packet has already become a dumping ground — seven different sub-messages are tunnelled through one type via an “origin byte,” which is the tell-tale symptom of having run out of message-ID space.

This document specifies **NavLink v2**: a MAVLink-class framing layer that keeps NavLink’s simplicity and CRC discipline but adds a 24-bit message-ID space, system/component addressing, sequence numbering and link statistics, a `CRC_EXTRA` message-signature scheme, payload-truncation for forward/ backward compatibility, an acknowledged command/parameter/mission service layer, toggleable AEAD security (per-frame signing and/or encryption), and a single message-catalog source of truth that generates the firmware, GCS, and Python codecs. It also defines the new packets the vehicle already needs (GPS, battery, position, parameters, missions, ESC telemetry, etc.) and a phased migration that runs v1 and v2 side-by-side so nothing breaks during the transition.

2. NavLink v1 — As-Built

2.1 Wire format

All multi-byte fields are little-endian. Header is 8 bytes; CRC trails the payload.

offset	size	field	
0	1	sync	= 0x56 ('V')
1	1	protocol_packet_type	= (packet_type << 4) (version & 0x0F)
2	1	length	= payload byte count (0..256)

3	1	device_id	= sender id (informational)
4	4	timestamp	= uint32 (unix secs / free-running)
8	N	payload	= 0..256 bytes
8+N	4	crc32	= CRC-32/MPEG-2 over bytes [0 .. 8+N)

Source of truth: `include/comm/comm_types.h:62` (`packet_t`), `include/variables.h:271` (sizes), `src/comm/serializer.c:22` (TX), `src/comm/deserializer.c:13` (RX state machine).

- **Version** = 0x1, packed in the *low* nibble of byte 1 (`comm_types.h:8`). 4 bits → 16 versions.
- **Packet type** = *high* nibble of byte 1. 4 bits → **16 message types**.
- **CRC** = CRC-32/MPEG-2: poly 0x04C11DB7, init 0xFFFFFFFF, no reflection, no final XOR (`software/src/core/crc.cpp:3`). Computed over header+payload, *excluding* the CRC field. The firmware additionally rejects a packet whose computed CRC is 0 (`deserializer.c:60`) — a quirk that throws away ~1 in 4 billion otherwise-valid frames.

2.2 Message catalog (v1, as-built)

Type	Name	Dir	Payload	Notes
0x0	HEARTBEAT	FC→GCS	0	~1 Hz
0x1	IMU_DATA_FULL	FC→GCS	40	10×f32: acc,gyr,mag,temp
0x2	IMU_DATA_COMPRESSED	FC→GCS	20	10×f16 deltas vs last full
0x3	COMMAND	GCS→FC	var	[cmd_id:2] [argc:1] [args:16]
0x4	ATTITUDE	FC→GCS	12	roll,pitch,yaw (deg)
0x5	RC_CHANNELS	FC→GCS	28	14×u16 ps
0x6	SYSTEM_STATUS	FC→GCS	var	multiplexed (see below)
0x7	LOG	FC→GCS	var	ASCII text
0x8	MOTOR_TELEMETRY	FC→GCS	16	4×f32

8 of 16 type slots are already used. `SYSTEM_STATUS` (0x6) is itself a sub-protocol, demultiplexed on `payload[0]` (“origin”):

Origin	Sub-message	Layout
0x01	CALIBRATION	[01] [type:u8] [f32] or [01] [type:u8] [3×f32]
0x02	HEALTH	[02] [pad] [tx_ovf:u32] [imu_drop:u32]
0x03	MOTOR	reserved
0x04	SYS_STATE	[04] [pad] [state:f32]
0x05	PID_ERROR	[05] [18] [18×f32] — 74-byte control-loop trace
0x06	PID_UPDATE	reserved
0x07	FLIGHT_MODE	[07] [pad] [mode:u8] [source:u8]

Commands (type 0x3) carry a second 16-bit ID space (`comm_types.h:52`): `CMD_CALIBRATE_IMU=0x0001`, `CMD_ARM=0x0002`, `CMD_DISARM=0x0003`, `CMD_SET_PID=0x000A`, `CMD_SET_GYRO_LPF=0x000B`, `CMD_SET_MOTOR_GEOMETRY=0x000C`

CMD_SET_FLIGHT_MODE=0x000D, plus an undocumented 0x0009 cancel-calibration handled by literal (`comm_processor.c:65`).

2.3 Framing & dispatch

- **FC RX** is a 4-state byte-at-a-time machine (SYNC→HEADER→PAYLOAD→CRC, `deserializer.c`), fed one byte per UART ISR (`serializer.c:76`). Valid packets land in a 3-deep array; full → drop counter. Resync on any malformed byte restarts at the next 0x56.
- **FC dispatch** is a hand-written `if/else` chain on packet type, then a nested `if/else` on `cmd_id` (`comm_processor.c:39`).
- **GCS RX** is a buffer-scan parser (`DroneProtocol.cpp:15`) that finds 0x56, validates version+length+CRC, emits a Qt signal per type, then a second `std::variant` switch in `PacketDecoder.cpp`.
- **Tools** re-implement the same framing+CRC a third time in `tools/autotune/protocol.py`.

2.4 Bandwidth (current load)

From `telemetry_task.c` at the ~166 Hz base tick: full IMU 1 Hz, compressed IMU 25 Hz, attitude 10 Hz, RC 10 Hz, motor 18 Hz, PID trace 18 Hz, status 2 Hz, log 15 Hz, heartbeat 1 Hz. The 74-byte PID trace at 18 Hz dominates (~1.5 kB/s); aggregate steady-state is roughly **3–4 kB/s**, comfortably inside a 115200-baud link (~11.5 kB/s) but with little headroom once GPS, battery and position streams are added.

3. Why v1 Doesn't Scale — Limitations

Ordered by how badly each blocks growth.

L1 — 4-bit message-ID space (hard ceiling)

16 types total, 8 left, and one (`SYSTEM_STATUS`) is already a nested demuxer hiding 7 more messages. Every new concept (GPS, battery, position, parameters, missions, gimbal, ESC, vibration, ...) either consumes a scarce top-level slot or gets tunnelled through `SYSTEM_STATUS`, which couples unrelated messages, defeats per-message rate control, and makes the GCS decoder branch on a magic byte. MAVLink has a 24-bit msgid space (16M). **This is the single most important limitation.**

L2 — No wire-format integrity across versions (CRC_EXTRA gap)

The CRC proves the *bytes* arrived intact, not that sender and receiver *agree on what those bytes mean*. If the FC adds a field to `ATTITUDE` and the GCS wasn't rebuilt, the GCS happily decodes 12 bytes with the new layout and shows garbage — the CRC still passes. Today this is “caught” only by exact-length checks in the GCS (`PacketDecoder.cpp`), which reject the whole message on any size change rather than tolerating it. MAVLink seeds the CRC with a per-message `CRC_EXTRA` hash of the field names/types so a layout mismatch *fails the CRC* and is detected, not mis-decoded.

L3 — No sequence numbers → packet loss is invisible

There is no per-stream counter anywhere in NavLink (only the unrelated vsim sim protocol has `seq_no`). The GCS cannot compute link quality, drop rate, or detect a stalled stream except via the 1 Hz heartbeat. Autotune and logging cannot tell a dropped sample from a slow one.

L4 — No real addressing / routing

`device_id` is a single byte, set by the sender, never used for filtering or routing (`comm_processor.c:44` just stores it). There is no component ID, so you cannot address “the gimbal on vehicle 2,” cannot run a companion computer + GCS on the same link, and cannot fan a message out across a multi-hop link (telemetry radio → USB → UDP). MAVLink's (`system_id`, `component_id`) pair plus routing tables are what make those topologies work.

L5 — No acknowledged delivery

Commands are fire-and-forget. `CMD_ARM`, `CMD_SET_PID`, etc. have no ACK, no result code, no retransmission. The autotuner *polls* `SYSTEM_STATUS` to infer whether a `SET_PID` took effect (`tools/autotune/`), which is racy and slow. There is no way to know a command was rejected vs lost.

L6 — Two parallel, inconsistent ID spaces

“Messages” use a 4-bit type; “commands” use a 16-bit `cmd_id` *inside* one type, with their own `argc/args` ABI. Parameters and missions, when they come, would need yet another scheme. The fix is to make commands *just messages* (each a typed message + `COMMAND_ACK`, §8.1), as params and missions also are — one ID space, one ABI.

L7 — No forward/backward field compatibility

Payload fields are fixed-offset and fixed-length; the decoder rejects any length change (L2). You cannot add a field to an existing message without a coordinated, simultaneous rebuild of FC + GCS + tools. MAVLink 2 allows appending fields and truncates trailing zero bytes on the wire, so new senders and old receivers interoperate.

L8 — Three hand-maintained codecs, no source of truth

The same framing, CRC, and per-message layout are coded by hand in C (firmware), C++ (GCS), and Python (tools). Drift between them is a recurring bug source (it is *exactly* the L2 failure mode). The fix is to generate all three bindings from one machine-readable dialect (NavLink v2 uses JSON — §9 Phase 0).

L9 — Weak resync, small RX buffers

Resync discards a single byte and rescans for `0x56` — fine at low error rates, pathological on a noisy radio where `0x56` appears in payload data. There is no length sanity bound beyond the 256 cap, and only a 3-packet RX ring (`INCOMING_PACKET_BUFFER=3`); a burst overflows it silently.

L10 — Miscellaneous

- 32-bit `timestamp` with ambiguous units (unix secs vs counter); ~4 s rollover if it is ever a ms counter. (*v2 removes it from the header and redefines the wall-clock field as `[s:20/ms:12]` from a synced epoch — §7, §8.6.*)
- `crc==0` rejected (`deserializer.c:60`).
- No authentication/signing — anyone on the link can arm the vehicle.
- No fragmentation: payload is hard-capped at 256 B; a future mission upload or log download must hand-roll chunking.
- No timestamping convention for cross-message alignment (each message carries the FC’s send time, not the sample time). (*v2: wall-clock time lives in payloads against a defined, GCS-synced epoch, and high-rate samples carry their own μ s fields — §7, §8.6.*)

4. Design Goals for v2

1. **Scale the ID space** to thousands of messages without tunnelling.
2. **Detect, never mis-decode**, version/layout drift (`CRC_EXTRA`).
3. **Measure link quality** (sequence numbers, per-link stats).
4. **Address & route** across systems, components, and transports.
5. **Acknowledge** commands, params, and transfers; make them reliable.
6. **Evolve messages** by appending fields without breaking old peers.
7. **One source of truth** generating C / C++ / Python codecs.

8. **Stay cheap:** ~12-byte header, table-driven CRC, no dynamic alloc on the FC hot path, fits the F4 and the existing UART budget.
9. **Migrate incrementally:** v1 and v2 coexist on one wire; no flag day.
10. **Optional confidentiality + integrity/auth** (AEAD: per-frame encryption and/or signing), runtime-toggable, for fielded vehicles.
11. **Synchronize time:** a defined wall-clock epoch plus a GCS-driven sync service, so payload timestamps are comparable across messages and against GCS time — no clock in the header.

Non-goals: compatibility with any existing protocol. NavLink v2 is its own, leaner frame. Where it resembles MAVLink that is *because the mechanism is load-bearing on its own merits* — CRC_EXTRA (semantic drift detection), a wide msgid space (kills the ceiling), append-and-truncate field growth — not because resembling MAVLink has any value here; if compatibility were the goal we would just run MAVLink. Where MAVLink’s choices are artifacts of its stateless, single-frame-self-describing history, we deliberately diverge — e.g. no per-frame `compat_flags` byte; capability negotiation at connect instead (§5). Both **authentication and encryption are in scope** as toggleable AEAD modes (§8.5) — confidentiality is no longer a non-goal. Still out of scope: TCP-style ordered reliability for telemetry (telemetry stays best-effort; only the service layers are acknowledged).

5. NavLink v2 — Frame Format

Fixed 10-byte header (unsigned), little-endian, CRC trailer.

offset	size	field	
0	1	sync	= 0x56 ('V') (same as v1; version byte demuxes - see below)
1	1	version	= 0x02 (v1 carries 0x_1 here; low nibble tells v1 from v2)
2	1	payload_len	= 0..255 (truncated length; see §7)
3	1	incompat_flags	(frame is undecodable if any unknown bit set)
4	1	seq	(per-(sysid,compid) rolling 0..255)
5	1	sysid	(1..255; 0 = broadcast/anonymous)
6	1	compid	(component within system)
7	3	msgid (u24, little-endian)	(0..16,777,215)
10	N	payload	= payload_len bytes (truncated, see §7)
10+N	2	checksum	= CRC-16/MCRF4XX seeded with CRC_EXTRA (see §6)
[sec]	6+N	secured trailer	= [timestamp:6][tag:N] replaces the CRC when incompat_flags & (IFLAG_SIGNED IFLAG_ENCRYPTED); see §8.5

Notes / rationale:

- **Sync 0x56 ('V')** is unchanged from v1 — it is Vayu’s own frame marker, not borrowed from any other protocol (an earlier draft used 0xFD, which is MAVLink2’s start byte; reusing it bought nothing once compatibility is a non-goal, §4). Because v1 and v2 now share the sync byte, the **version byte** (offset 1) — not the sync byte — is what demultiplexes them on one stream during migration (§9).
- **version = 0x02.** v1 packs (`packet_type << 4`) | **version** into byte 1 with **version = 0x1**, so *every* v1 frame has a low nibble of 1 there; v2 sets the whole byte to 0x02. A receiver reads 0x56, then branches on byte 1: low-nibble 1 → v1, value 0x02 → v2, anything else → resync. No v1 frame can collide (its low nibble is always 1; 0x02 never appears), and no v2 frame can be mistaken for v1. The byte doubles as a real, self-documenting protocol version (v3 → 0x03) — exactly the implicit job the distinct magic byte was doing, now made explicit. This is the byte reclaimed from dropping `compat_flags`, re-spent on something load-bearing.
- **incompat_flags** is the forward-compat kill switch: if a receiver sees a bit it doesn’t understand, it **MUST** drop the frame, because such a bit signals a change to the frame’s *physical structure* that must be known before the frame can even be located or its CRC verified (e.g. a signature trailer changes where the frame ends). `IFLAG_SIGNED = 0x01` and `IFLAG_ENCRYPTED = 0x02` (§8.5) are the first — both swap the CRC for a [timestamp][tag] trailer; `IFLAG_CRC32 = 0x04` (§8.4) and a future

IFLAG_FRAGMENTED = 0x08 are the obvious next ones. It is a **full byte, not a nibble**: this is the one header field where exhausting the bits forces a frame-format bump (the same trap that v1's 4-bit msgid fell into), so it is the one field that earns its headroom.

- **There is no compat_flags field.** A per-frame byte of advisory, layout-neutral bits is the wrong home for link-wide metadata that changes at most once per session. NavLink controls both endpoints and is *not* stateless per frame, so anything advisory is **negotiated once at connect** — via HEARTBEAT capability bits plus an optional CAPABILITIES message for the full list (§10.1) — rather than re-sent in every header. This is a deliberate departure from per-frame-flag protocols: a capabilities exchange carries typed *fields* (protocol version, supported *incompat* bits, msgid ranges, key IDs), not just opaque bits, and both ends *know* what the other supports instead of setting a bit and hoping it is ignored.
- **CRC-16** (not CRC-32). At our frame sizes CRC-16/MCRF4XX gives ample Hamming distance, halves the trailer, and — critically — is the field that carries CRC_EXTRA. (We keep an *optional* CRC-32 mode for large bulk transfers behind an *incompat* flag — it changes the trailer width, so a receiver must know about it to parse the frame at all; see §8.4.)
- **Header is 10 bytes** vs v1's 8 — two extra bytes buy the entire addressing + sequencing + 24-bit-ID upgrade *and* an explicit version byte. For a 12-byte attitude message the overhead goes 8→10; negligible.
- **No timestamp in the header.** v1 spent 4 bytes/frame on an ambiguous send- time clock (L10). v2 drops it entirely: wall-clock time rides only in the payloads that need it (HEARTBEAT always), with a defined [s:20|ms:12] encoding (§7) maintained by an explicit sync service (§8.6). Net header cost vs v1 is *negative* once that 4-byte field is gone.

5.1 Sizes & limits

Quantity	v1	v2
Message IDs	16	16,777,216 (top bit: core / vendor)
Max payload	256	255 (single frame); unbounded via §8.4 chunked transfer
Header bytes	8	10 (+13 if signed)
CRC	32-bit	16-bit + CRC_EXTRA seed
Addressing	1 (informational)	sysid + compid
Sequence numbers	none	per-(sysid,compid)
Versions	4-bit field	version byte + incompat_flags + capability negotiation

6. CRC_EXTRA — Version-Safe Decoding

For each message, codegen computes a 1-byte CRC_EXTRA by running CRC-16/MCRF4XX over the message *name and the type+name of every non-extension field in declaration (index) order* — which is also wire order, since NavLink does not reorder fields (§7) — then folding the 16-bit result to 8 bits ((crc ^ (crc >> 8)) & 0xFF). This is the MAVLink algorithm.

The frame checksum is computed as:

```

crc = crc16_mcrf4xx_init()
crc = crc16_update(crc, header_bytes[1 .. 9]) // everything after sync (version..msgid)
crc = crc16_update(crc, payload[0 .. payload_len])
crc = crc16_update(crc, CRC_EXTRA[msgid]) // the per-message seed byte
checksum = crc

```

Effect: if the sender's and receiver's definition of message *M* differ in field count, order, type, or name, their CRC_EXTRA differ, the checksum **fails**, and the frame is dropped instead of silently mis-decoded (fixes **L2**). Same bytes, different meaning → detected.

The receiver keeps a `msgid` → `CRC_EXTRA` table (generated). Unknown `msgid` → unknown `CRC_EXTRA` → frame is dropped as un-decodable, but framing is *not* lost (length is in the header, so the parser skips exactly the right number of bytes and stays in sync — fixes part of **L9**).

7. Field Encoding & Payload Truncation (forward/backward compat)

NavLink v2 adopts MAVLink2's *growth* rules (extensions, truncation) but deliberately **drops MAVLink's wire field-reordering**:

1. **Declaration order on the wire; packed structs.** MAVLink emits fields largest-type-first so that even an *unpadded* C struct would align to the wire — at the cost that wire-order ≠ source-order, its single most confusing codegen behaviour. NavLink instead emits fields in **declaration (index) order** and generates **packed** structs (`__attribute__((packed))`, §9), so the struct already overlays the wire byte-for-byte with no compiler padding. Both ends are little-endian, so the codec serialises a whole payload with a single `memcpy` into the packed wire struct (§9), and the wire is exactly the order you read in the dialect. FC/GCS application code works on a separate natural-aligned copy of the message (§9), so it never reads fields out of the packed struct directly; where the wire form itself wants alignment, the dialect author adds an explicit `pad` field — **padding is stated, never compiler-inserted**.
2. **Extension fields.** Fields added after a message ships are marked `<extension>` and always serialize *after* all original fields, in declaration order (no resorting). `CRC_EXTRA` is computed over *only the original* fields, so adding extensions does **not** change `CRC_EXTRA` → old and new peers still pass each other's CRC.
3. **Trailing-zero truncation.** The sender drops trailing all-zero bytes and reports the shortened `payload_len`. The receiver zero-fills the tail back to the message's known length before decoding. So:
 - *New sender* → *old receiver*: extension bytes are simply beyond what the old receiver reads; it decodes the prefix it knows. [OK]
 - *Old sender* → *new receiver*: short payload is zero-extended; new fields read as 0 (their defined "absent" value). [OK]

This fixes **L7**: you can append `gps_fix_type` to a position message years later and never coordinate a flag-day rebuild. (On encrypted frames, truncation runs on the plaintext *before* encryption — order pinned in §8.5.)

7.1 Time fields

Wall-clock timestamps use one fixed encoding — a 32-bit `[seconds:20 | ms:12]` word (little-endian, seconds in the high 20 bits):

- **20-bit seconds** = an integer offset from the synchronized reference epoch `T0` the GCS established (§8.6), *not* an absolute Unix time. Range is 2^2 s = **291 h**, which dwarfs the 5 h periodic-resync interval, so it never rolls over within a session.
- **12-bit milliseconds** = the 0..999 sub-second remainder. This is where the FC's residual clock error relative to `T0` shows up; the resync schedule keeps that error inside the 1 ms LSB. (10 bits would cover 0..999; the 2 spare high bits are reserved — one as a `TIME_STALE` flag the FC sets before its first sync and after any clock discontinuity, so the GCS never reads an unsynced stamp as synced.)

This deliberately keeps v1's 32-bit width while removing its ambiguity (L10): the epoch is *defined*, not guessed. High-rate sensor data keeps its own microsecond field (`sample_time_us`, §10.2) for intra-vehicle alignment that needs finer-than-ms resolution; the `[s:20|ms:12]` form is the GCS-referenced wall-clock stamp, carried by `HEARTBEAT` and any other message that needs it.

7.2 Numeric types & fixed-point fields

Every field declares its wire type **explicitly** in the dialect (§9), from a fixed enum — `u8/i8/u16/i16/u32/i32/u64/i64/f32/f64` — never inferred; codegen rejects a missing or unknown type.

For decimal quantities, use **fixed-point integers** wherever float precision actually fails — most importantly **position**. An **f32** carries only ~ 7 significant digits *and* its precision is relative, so a global latitude (180.0000001) resolves to no better than ~ 1 m, worse far from the origin. Storing **degrees $\times 1e7$ in an i32** (“degE7”) gives a *uniform* ~ 1.1 cm everywhere — this is why **GPS_RAW_INT**/**GLOBAL_POSITION_INT** (§10.3) are integer types. The scale and unit are declared on the field so the bits are unambiguous:

```
{ "index": 1, "name": "lat", "type": "i32", "scale": 1e7, "unit": "deg" }
```

The scale is chosen **per field to fit range $\times$ resolution into the integer width** — it is *not* a blanket “ $\times 1e7$ on every float”:

- **i32** spans $\pm 2.147 \times 10^9$, so $\times 1e7$ leaves a usable range of only ± 214.7 — perfect for $\pm 180^\circ$ lat/lon, but it would *overflow* an altitude in metres. Altitude therefore uses **mm ($\times 1e3$)** $\rightarrow \pm 2,147$ km at 1 mm resolution.
- Small, bounded quantities stay **f32/f64** — attitude angles ($\pm$ rad), body rates, temperatures, voltages — where 7/15 digits over a narrow range already beat the sensor, and fixed-point would add complexity for no gain. A uniform $\times 1e7$ would also overflow large-range fields and cannot represent **f64**.

(This is also why the dialect’s own JSON literals stay exact: an integer scale like **1e7** and any default under 2^3 is represented exactly as a JSON double; write 64-bit constants as hex strings, §9.)

8. Service Layers (the “microservices”)

Telemetry stays best-effort streaming. Everything that needs *reliability* or *transactions* rides standard request/response message patterns, like MAVLink’s microservices. All of these are just ordinary v2 messages — no second ID space (fixes **L6**).

8.1 Command service — typed commands, no float box

MAVLink forces every command’s parameters into seven **float32** slots (**COMMAND_LONG**). A **float32** cannot hold an exact **int32** past 2^2 , so any command needing integer precision spawned the parallel **COMMAND_INT** — a permanent “which one do I use?” fork — and every command pays 28 bytes of params whether it uses them or not. NavLink drops both.

A command is an ordinary typed message in the reserved command range (§10.7), defined in the dialect like any other — its parameters are real, individually-typed, truncatable fields (**u8**, **i32**, **f32**, **char[]**, fixed-point, whatever the command actually needs). No float straitjacket, no **LONG/INT** split, no wasted bytes, and the arguments are self-describing in the dialect instead of positional floats. Each command message opens with a shared 3-byte header (a dialect template):

```
{ target_sys:u8, target_comp:u8, req_seq:u8 } // followed by the command's typed fields
```

req_seq is a small rolling id the GCS chooses so the reply can be correlated. One generic ack answers any command:

- **COMMAND_ACK** { **command**:u24, **req_seq**:u8, **result**: enum{ACCEPTED, TEMPORARILY_REJECTED, DENIED, UNSUPPORTED, FAILED, IN_PROGRESS}, **progress**:u8, **result_param2**:i32 } — **command** is the acked message’s msgid, **req_seq** matches the request.

The current **CMD_ARM**/**DISARM**/**SET_PID**/**SET_GYRO_LPF**/**SET_MOTOR_GEOMETRY**/ **SET_FLIGHT_MODE**/**CALIBRATE_IMU** each become a typed command message — e.g. **CMD_SET_PID** { ...header..., **axis**:u8, **kp**:f32, **ki**:f32, **kd**:f32 } — and each returns a **COMMAND_ACK** (fixes **L5**: the autotuner waits for **ACCEPTED**/**IN_PROGRESS**/**FAILED** instead of polling).

8.2 Parameter service — typed values, transactional dumps

Two MAVLink faults to avoid. First, MAVLink sends every parameter as a **float32** with a side **param_type**, **bit-reinterpreting** integers through the float field — a footgun. Second, its list dump is

non-transactional: a lost PARAM_VALUE mid-stream is undetectable and a PARAM_SET has no reliable confirmation, so GCSs paper over it with polling and timeouts. NavLink fixes both:

- **Typed values.** PARAM_VALUE { param_id:char[16], index:u16, count:u16, generation:u32, type:u8, value } — value is encoded in its *real* type (u8..i64/f32/f64, 8 B, truncated to width), never a bit-cast float. Each parameter’s type / unit / scale / range / default come straight from the dialect (the same single source of truth, §9), so the table is generated, not hand-listed.
- **Transactional dump.** Every value carries a **generation** (bumped on any PARAM_SET). The GCS has a complete, consistent set only when it holds all count indices under one generation; if generation changes mid-dump it re-syncs, and any missing index is re-requested by PARAM_REQUEST_READ {index}.
- **Confirmed writes.** A PARAM_SET is answered by the updated PARAM_VALUE carrying the new generation, so “did my set take?” is *answered*, not inferred — closing the MAVLink param race.

PARAM_REQUEST_LIST / PARAM_REQUEST_READ { param_id:char[16] | index } / PARAM_SET { param_id, type, value } round it out. This generalises the ad-hoc PID/LPF/geometry setters into a discoverable, typed, persisted table the GCS can enumerate and edit without bespoke per-command UI, and the autotuner reads/writes gains by name with confirmation.

8.3 Mission service — waypoints / geofence / rally

- MISSION_COUNT, MISSION_REQUEST_INT { seq }, MISSION_ITEM_INT { seq, frame, command, x, y, z, ... }, MISSION_ACK, MISSION_CURRENT, MISSION_CLEAR_ALL. Acknowledged, seq-numbered transfer (the canonical reliable-transfer pattern, also reused for any list upload).

Like params (§8.2), a mission carries a **generation** and **commits atomically**: the FC applies the new mission only once all MISSION_COUNT items have arrived and validated, and MISSION_ACK returns the committed generation. This closes MAVLink’s mission edge cases where a dropped item mid-upload can leave a half-applied plan — a partial upload here simply never commits.

8.4 Bulk transfer (logs, large blobs) — fixes the 255-byte cap

- FILE_TRANSFER { session, offset:u32, len:u8, data[] } with windowed ACKs, plus an optional CRC-32 frame mode (an incompat flag, since it changes the trailer width) for the big chunks. Used for SD-card log download (0:v_nav.bin) and param/calibration blob upload — replaces the implicit “application chunks it somehow” gap (L10).

8.5 Security — authentication & encryption (toggleable)

NavLink v2 supports per-frame **authentication** and **confidentiality**, both runtime-toggleable and selected through incompat_flags. Two design rules keep it from becoming the usual embedded-crypto footgun:

1. **Use AEAD — don’t bolt encryption onto a separate MAC.** Authenticated Encryption with Associated Data (ChaCha20-Poly1305, or AES-GCM/CCM) gives confidentiality *and* integrity/authenticity in **one** pass, with no encrypt-then-MAC ordering to get wrong. “Encryption” and “signing” are not two separate mechanisms here — they are two *modes* of one AEAD construction.
2. **Header cleartext, payload encrypted.** Addressing (sysid/compid/msgid/ len/seq) must stay readable to route and frame the packet, so it is the AEAD **associated data** — authenticated but not hidden. Only the payload is encrypted. The auth **tag** covers header ciphertext and *supersedes* the CRC-16, so the CRC field is dropped on secured frames (the tag is far stronger), clawing back 2 bytes.

Modes (incompat_flags, both default-off):

- IFLAG_SIGNED = 0x01 — authenticate only; payload stays cleartext. The cheapest way to stop a stranger arming the vehicle while leaving telemetry readable.
- IFLAG_ENCRYPTED = 0x02 — full AEAD; payload encrypted *and* authenticated (implies authentication).

Because the flags are per-frame you can mix: leave high-rate telemetry cleartext (its confidentiality is low-value) and require `IFLAG_ENCRYPTED` — or at least `IFLAG_SIGNED` — on the command/arm path (high-value). A vehicle can be configured to **reject** an unsecured `CMD_ARM`.

Secured trailer (replaces the CRC when either flag is set):

[timestamp : 6][tag : N] N = 16 (full) or 8 (truncated, weaker auth)

- **timestamp** — a 48-bit **wall-clock-anchored** stamp (100 μ s ticks since the synced epoch T0, §8.6), strictly monotonic per (sysid, compid, direction). It does double duty: (a) **replay protection** — the receiver accepts a frame only if its stamp is *both* inside an acceptance window [`now - skew`, `now`] against its own synced clock *and* greater than the last stamp accepted from that peer; (b) **nonce material** (below).
- **tag** — the AEAD authentication tag over (header | ciphertext).

Replay survives reboot — **because the anchor is the synced clock, not a counter**. A power-cycle cannot rewind the stamp: on reconnect the FC re-syncs time (§8.6), so any frame captured from a past session carries a stamp far outside the window and is rejected — nothing need be persisted to flash. The hard rule: **the FC must complete a time sync before it accepts any secured or command frame; if its clock and the GCS reference disagree beyond skew it is “unsynced” and accepts nothing** until a fresh sync succeeds (§8.6). Time is the gate.

Nonce. The 96-bit AEAD nonce is built deterministically from fields the receiver already holds — never transmitted in full, never random:

nonce[12] = dir(1) sysid(1) compid(1) seq(1) timestamp(6) 0x0000(2)

dir is 0 for FC→GCS, 1 for GCS→FC (the two directions never share a nonce under one key); sysid/compid/seq come from the cleartext header, timestamp from the trailer. Uniqueness holds as long as (timestamp, seq) never repeats for a (dir, sysid, compid) — guaranteed because timestamp is monotonic and a 100 μ s tick is finer than 256 frames of seq rollover at any real rate. This construction, not randomness, is what makes nonce reuse (which breaks AEAD catastrophically) impossible.

Order of operations: truncate, then encrypt. Trailing-zero truncation (§7) runs on the **plaintext** — the sender drops trailing-zero plaintext, sets `payload_len`, then encrypts exactly those bytes (a stream cipher preserves length). The receiver decrypts, then zero-fills back to the message’s known length. `payload_len` rides in the header, which is authenticated as AAD, so it cannot be tampered to change the fill.

Cipher. Default **ChaCha20-Poly1305**: fast and constant-time in *software* on the Cortex-M4, needs no crypto peripheral — which matters, because the common F405/F407 have **no** hardware AES (only F415/417/437/439 ship the CRYPT block). Where hardware AES exists, **AES-GCM/CCM** is an equally valid build option. **Do not hand-roll the primitive** — use a vetted, small library (e.g. Monocypher: ChaCha20-Poly1305 + X25519, public-domain, embedded-sized).

Keys. The **pre-shared key** lives in a file on the FC’s SD card (`0:navlink.key`), read once at boot and never transmitted; the GCS holds the matching key in its config. Provisioning is genuinely out of band — you write the card directly (card reader / USB mass-storage), so the secret never crosses the radio. `CAPABILITIES.key_id` carries a **non-secret** key identifier so both ends confirm they hold the same key at connect (a mismatch is reported, not left to silent decrypt failures). **Key rotation** is permitted only over USB or an already-secured link — never bootstrapped over an unsecured radio. A later option is an **X25519 (ECDH) handshake at connect** for per-session keys and forward secrecy (the device’s static public key provisioned the same way, optionally pinned in the GCS), negotiated via `CAPABILITIES` — heavier, opt-in, not required for the first cut.

Turning it off. Everything here is gated by a build flag (`ENABLE_NAVLINK_CRYPT0`) and advertised in `CAPABILITIES`; with crypto off, the frames are exactly the plain CRC-16 form of §5. Security is never silently assumed: a peer that sets a security flag a receiver did not negotiate is dropped (the flags are `incompat`, §5), so a downgrade to cleartext can’t pass unnoticed.

8.6 Time synchronization — the epoch behind every payload timestamp

There is no clock in the frame header (§5). Wall-clock time lives only in the payloads that need it, encoded as [s:20|ms:12] offsets from a reference epoch T0 (§7.1). T0 is established and maintained by the GCS through an explicit sync service — the FC never invents its own wall-clock:

The sync is a **GCS-initiated round trip** that compensates for link latency — necessary, not optional, because one-way delay at 115200 baud is 1.4 ms for a small frame, *comparable to the 1 ms LSB*: a one-way push would bake that delay straight into the epoch. The exchange records four NTP-style timestamps:

- **TIME_REFERENCE** { epoch_unix_s:u32, gcs_send_us:u64 } — **GCS→FC**, stamped with the GCS send time **t1** (gcs_send_us).
- **TIME_REFERENCE_ACK** { gcs_send_us, fc_rcv_us, fc_send_us } — **FC→GCS**, echoing **t1** and adding the FC receive time **t2** (fc_rcv_us) and FC send time **t3** (fc_send_us). The GCS notes its own receive time **t4**.

From the four stamps the GCS computes, exactly as NTP does:

```
rtt      = (t4 - t1) - (t3 - t2)          // round trip minus FC processing
delay    = rtt / 2                        // one-way link latency
offset   = ((t2 - t1) + (t3 - t4)) / 2    // FC-clock vs GCS-clock skew
```

The GCS then pushes the **latency-corrected** T0 (a second **TIME_REFERENCE** carrying epoch_unix_s adjusted by **delay**), and the FC latches its local tick counter to it and thereafter emits **now** - T0. Subtracting **delay** is what keeps the synced error inside the 1 ms LSB rather than ~1.4 ms off.

Sync schedule:

1. **Startup sync** — at link establishment, before any FC stamp is trusted. Until the first **TIME_REFERENCE** lands, the FC flags its time fields **TIME_STALE** (§7.1).
2. **Periodic resync** — every **5 h** of continuous operation, to bound accumulated crystal drift. A few-ppm TCXO drifts only a handful of ms over 5 h, keeping the error inside the 1 ms LSB between resyncs.
3. **Emergency resync** — the GCS continuously compares incoming FC stamps to its own clock; if the estimated error crosses a configured threshold it pushes an out-of-schedule **TIME_REFERENCE** immediately.

Time is the security gate. On contact with a GCS the FC runs the startup sync **first**. Until it succeeds — and whenever the FC's clock and the GCS reference disagree beyond the acceptance **skew** — the FC is *unsynced*: it flags its own time fields **TIME_STALE** (§7.1) and **rejects every secured and command frame** (§8.5). A secured link therefore cannot even begin before time agrees, which is exactly what lets the synced wall-clock serve as the anti-replay anchor (no persisted counter, reboot-safe).

Because every resync re-establishes T0 near zero, the 20-bit seconds field is never close to its 291 h ceiling — the schedule and the field size are deliberately matched.

9. Migration Plan (v1 v2 coexistence)

The two protocols share one UART **and the same 0x56 sync byte**. A **dual parser** demultiplexes them on the **version byte** (offset 1): every v1 frame has a low nibble of 1 there, every v2 frame carries 0x02 (§5). So we migrate message by message with no flag day.

Phase 0 — Catalog & codegen (no wire change)

- Author `navlink/dialect.json` as the **single source of truth**: every v1 message re-described with field names/types, plus the new messages from §10. Mark which (**sysid**, **compid**) each is for. JSON over XML — it is more readable and the generator parses it natively — under three rules that make it safe as an ABI definition:

1. **Fields are an ordered array, never a key→type object.** JSON object keys are spec-unordered; wire order is load-bearing, so fields must be a list.
2. **Every field carries an explicit index.** Order is *stated*, not inferred from array position — reordering lines in the file cannot silently change the ABI, and CRC_EXTRA is computed deterministically from the indices. Codegen **validates** them (unique, contiguous, no gaps; extensions continue the sequence) and hard-fails on any violation.
3. **Annotations live in a doc string** per message/field (strict JSON has no comments); codegen emits them as docstrings / // comments in the generated C/C++/Python.
4. **Every field states its wire type explicitly**, from the fixed enum `u8/i8/u16/i16/u32/i32/u64/i64/f32/f64/c` — never inferred. Decimal quantities use fixed-point where float precision fails: an integer type plus an explicit **scale** and **unit** (e.g. lat as `i32 × 1e7 deg`). §7.2 covers when to use fixed-point vs float and how the scale is bounded.

```
{
  "messages": [
    {
      "msgid": 52, "name": "ATTITUDE_EULER", "replaces": "0x4",
      "doc": "Euler attitude + body rates, NED.",
      "fields": [
        { "index": 0, "name": "roll",      "type": "f32", "unit": "rad", "doc": "NED roll" },
        { "index": 1, "name": "pitch",    "type": "f32", "unit": "rad" },
        { "index": 2, "name": "yaw",      "type": "f32", "unit": "rad" },
        { "index": 3, "name": "rollspeed", "type": "f32", "unit": "rad/s" },
        { "index": 4, "name": "pitchspeed", "type": "f32", "unit": "rad/s" },
        { "index": 5, "name": "yawspeed",  "type": "f32", "unit": "rad/s" }
      ]
    }
  ]
}
```

Wire order (= declaration order, §7) and CRC_EXTRA (§6) are *derived* from this by codegen; the explicit **index** is the canonical input both come from. Write any 64-bit constants as hex strings (JSON numbers are doubles, exact only to 2^{-3}). Ship a JSON Schema for the dialect and check it in CI to catch duplicate msgids / indices and malformed types before generation.

- Write a small generator (`tools/navlink/generate.py`) emitting:
 - `include/comm/navlink_msgs.h` + `src/comm/navlink_msgs.c` (C structs, pack/unpack, CRC_EXTRA table) for firmware,
 - `software/src/protocol/NavlinkMsgs.{h,cpp}` for the GCS,
 - `tools/autotune/navlink_msgs.py` for tools.
- **A generated dispatcher.** Since every frame is decoded in the same FC, the dispatch belongs in the generated codec, not in hand-written branching. Codegen emits a `msgid → { decode, handler-slot }` table (C for the FC, the equivalent for the GCS) that decodes each frame into its aligned struct and calls the registered typed handler — `on_CMD_SET_PID(const cmd_set_pid_t*)`, etc. This replaces the hand-written `if/else` on packet type *and* the nested `if/else` on `cmd_id` (§2.3), and the GCS's `std::variant` switch (`PacketDecoder.cpp`): the application only fills in handler bodies, lookup is $O(1)$, and codegen can flag any `msgid` that has no handler so a new message can't be silently dropped.
- **Two representations per message, bridged by a struct copy.** Codegen emits
 - (a) a **packed wire struct** (`__attribute__((packed))` / `#pragma pack(1)`) laid out byte-for-byte like the wire (§7), and (b) a natural-aligned **application struct** for FC/GCS code. The codec moves the *whole payload* between the UART/socket buffer and the packed struct with a **single memcpy** — telemetry parsing therefore only ever touches the compact, raw, unaligned form (saves bandwidth; nothing to align or HardFault-debug, since no unaligned *typed pointer* is ever formed). Code that needs padded, fast, address-able fields converts to/from the aligned struct via generated field-wise `to_aligned()` / `from_aligned()` helpers — and this conversion is **lazy**: pure forward/log/relay paths stay on the packed struct and skip it, so the field-wise cost is paid only where the message is actually computed on. The whole-struct **memcpy** is valid because wire and

both hosts are little-endian and floats are IEEE-754 on both ends; the field-wise converters are the single place any future byte-swap would live. The only padding that ever appears on the wire is an explicit `pad` field in the dialect.

- This alone kills **L8** even before the new frame ships, by deleting the three hand-written codecs.
- Deliverable: generated v1 codec is byte-identical to today's output (golden-vector test against captured `v_nav.bin`).

Phase 1 — v2 framing library, behind a build flag

- Implement `navlink_v2_*` pack/parse in firmware + a `NavlinkV2Parser` in the GCS, selected by the version byte after the shared `0x56` sync. The receiver runs a **dual parser**: byte1 low-nibble `1` → v1 path, byte1 `0x02` → v2 path. Unknown/!sync → resync.
- Add `seq`, `sysid`, `compid`, CRC-16+CRC_EXTRA, the `incompat_flags` byte, and the HEARTBEAT/CAPABILITIES capability exchange (§10.1).
- Add link-stats accounting (drops via `seq` gaps) on the GCS.

Phase 2 — Dual-emit telemetry

- The FC emits each telemetry stream in **both** v1 and v2 (gated by `ENABLE_NAVLINK_V2`), or v2-only on a build flag, during bring-up. The GCS prefers v2 when present. Validate field-for-field parity on real flights/ SITL before removing the v1 emit.
- Move `SYSTEM_STATUS`'s seven tunnelled sub-messages to **first-class v2 msgids** (HEALTH, SYS_STATE, FLIGHT_MODE, PID_ERROR/CONTROL_TRACE, CALIBRATION_PROGRESS). This retires the origin-byte demuxer (**L1**).

Phase 3 — Command/param/mission services

- Replace type-0x3 `COMMAND` dispatch (`comm_processor.c`) with typed command messages + `COMMAND_ACK` (§8.1). Keep a thin shim that maps the legacy `cmd_ids` onto the new command msgids so the autotuner can switch over without a big-bang change.
- Stand up `PARAM_*` over the existing `pid.bin/cal.bin` persistence.
- Update `tools/autotune/protocol.py` to wait on `COMMAND_ACK`/`PARAM_VALUE` instead of polling `SYSTEM_STATUS` (**L5**).

Phase 4 — Retire v1

- Once SITL + a real flight confirm v2 parity, drop the v1 emit and the v1 parse path. Bump `incompat_flags` baseline. Keep the v1 decoder in the GCS only for reading old logs.

Code touch-points (for sizing)

- Firmware: `include/comm/comm_types.h`, `src/comm/serializer.c`, `deserializer.c`, `comm_processor.c`, `telemetry_task.c`, `variables.h`.
- GCS: `software/src/protocol/DroneProtocol.{cpp,h}`, `PacketDecoder.{cpp,h}`, `software/src/core/Types.h`, `crc.cpp`.
- Tools: `tools/autotune/protocol.py` (+ generated module).
- New: `navlink/dialect.json` (+ its JSON Schema), `tools/navlink/generate.py`, generated codec files in all three trees.

10. Proposed Message Catalog (v2)

The 24-bit msgid space (16,777,216 ids) is split on its **top bit**:

- `0x000000-0x7FFFFFF` (0 – 8,388,607) — **core / FC-owned**: the official NavLink dialect, versioned with the firmware. Only this tree assigns ids here.

- 0x800000–0xFFFFFFFF (8,388,608 – 16,777,215, top bit set) — **vendor / experimental**: out-of-tree dialects, third-party payloads, private extensions. Core never assigns in this half, so a vendor picks freely and can never collide with a future official message (MAVLink’s “msgid-squatting” problem, §13, designed out). (msgid & 0x800000) != 0 vendor.

The core half is laid out in **generously spaced blocks** so every category has room to grow and large gaps remain for the future. IDs below are illustrative — codegen assigns final values within each block.

Range (dec)	hex	Block	§
0–255	0x000000– 0x0000FF	Core / system	10.1
256–1023	0x000100– 0x0003FF	<i>reserved — core-system growth</i>	
1024–2047	0x000400– 0x0007FF	Sensors / state	10.2
2048–3071	0x000800– 0x000BFF	Navigation / position	10.3
3072–4095	0x000C00– 0x000FFF	Power / propulsion	10.4
4096–8191	0x001000– 0x001FFF	Peripherals / payload	10.6
8192–12287	0x002000– 0x002FFF	Commands (typed)	10.7
12288–16383	0x003000– 0x003FFF	Services (param/mission/file/cal)	10.5
16384–65535	0x004000– 0x00FFFF	<i>reserved — near-term core</i>	
65536–8388607	0x010000– 0x7FFFFFFF	<i>deep future core reserve (~8.3M ids)</i>	
8388608–16777215	0x800000– 0xFFFFFFFF	Vendor / experimental	—

(Subsection numbers 10.1–10.7 below are doc order; the table above is the authoritative ascending allocation.)

10.1 Core / system (0–255) — port of today

msgid	message	replaces	key fields
0	HEARTBEAT	0x0	type, autopilot, base_mode, system_status, nav_state, capabilities (bitmask), timestamp [s : 20 ms : 12]
1	SYS_STATUS	0x6/04	sensors_present/enabled/health bitmasks, load, voltage
2	SYSTEM_HEALTH	0x6/02	tx_overflow, imu_drop, log_wrap, cpu_load
3	FLIGHT_MODE	0x6/07	mode, source
4	STATUSTEXT	0x7	severity (enum), text[50], id, chunk_seq

msgid	message	replaces	key fields
5	COMMAND_ACK	0x3	acks any command by command msgid + req_seq — §8.1
6	(reserved)	—	no COMMAND_LONG/COMMAND_INT: commands are typed messages in the command range (§8.1, §10.7)
7	(reserved)	—	—
8	PING	—	seq, target — RTT/link probe
9	CAPABILITIES	—	protocol_version, incompat_supported (bitmask), msgid_ranges, sec_modes (none/sign/encrypt), key_id — exchanged once at connect; replaces a per-frame compat_flags byte (see §5, §8.5)
10	TIME_REFERENCE	—	epoch_unix_s, gcs_send_us — GCS→FC, sets the sync epoch T0 (§8.6)
11	TIME_REFERENCE_ACK	—	gcs_send_us, fc_rcv_us, fc_send_us — round-trip stamps for RTT/latency- compensated T0 (§8.6)

10.2 Sensors / state (1024–2047) — port + extend

msgid	message	replaces	key fields
1024	IMU_RAW	0x1	acc[3], gyr[3], mag[3], temp, sample_time_us
1025	IMU_COMPRESSED	0x2	f16 deltas + ref seq
1026	ATTITUDE_EULER	0x4	roll, pitch, yaw, rollspeed, pitchspeed, yawspeed
1027	ATTITUDE_QUATERNION	—	q[4], rollspeed, pitchspeed, yawspeed (NED-correct; see HUD work)
1028	RC_CHANNELS	0x5	18×u16, rssi, count
1029	MOTOR_TELEMETRY	0x8	per-motor cmd[8]
1030	CONTROL_TRACE	0x6/05	the 18-float PID/loop trace, first-class

msgid	message	replaces	key fields
1031	VIBRATION	—	vib_x/y/z, clip counts
1032	SCALED_PRESSURE	—	abs/diff pressure, temp (baro)

10.3 Navigation / position (2048–3071) — new, needed soon

msgid	message	key fields
2048	GPS_RAW_INT	fix_type, lat, lon, alt, eph, epv, vel, cog, satellites
2049	GLOBAL_POSITION_INT	lat, lon, alt, rel_alt, vx, vy, vz, hdg
2050	LOCAL_POSITION_NED	x, y, z, vx, vy, vz
2051	HOME_POSITION	lat, lon, alt
2052	ALTITUDE	alt_monotonic, alt_amsl, alt_local, alt_terrain
2053	ESTIMATOR_STATUS	flags, pos/vel/hgt ratios

Encoding (§7.2): `lat/lon` are `i32` degE7 ($\times 1e7$, ~ 1.1 cm), `alt/rel_alt` are `i32` millimetres ($\times 1e3$); velocities are `i16` cm/s. Float would lose >1 m on global coordinates — hence the integer types.

10.4 Power / propulsion (3072–4095) — new, needed soon

msgid	message	key fields
3072	BATTERY_STATUS	id, voltages[10], current, consumed_mAh, remaining%, temp
3073	ESC_TELEMETRY	per-esc rpm, voltage, current, temp
3074	POWER_STATUS	Vcc, Vservo, flags

10.5 Services (12288–16383)

msgid	message	service
12288–12299	PARAM_REQUEST_LIST / READ / VALUE / SET	§8.2
12300–12311	MISSION_COUNT / REQUEST_INT / ITEM_INT / ACK / CURRENT / CLEAR_ALL	§8.3
12312–12319	FILE_TRANSFER (logs, blobs)	§8.4
12320–12329	calibration progress/instructions (port of 0x6/01)	—

10.6 Payloads / peripherals (4096–8191)

GIMBAL_DEVICE_ATTITUDE_STATUS, CAMERA_TRIGGER, OBSTACLE_DISTANCE, COLLISION, ADSB_VEHICLE, TUNNEL (vendor-specific passthrough), DEBUG_VECT / NAMED_VALUE_FLOAT (ad-hoc dev telemetry without a schema change — useful during autotune experiments).

10.7 Commands (8192–12287) — typed, first-class

Each command is its own typed message (§8.1), not a 7-float envelope; all open with the { `target_sys`, `target_comp`, `req_seq` } header, then typed fields.

msgid	command	typed params (after header)
8192	<code>CMD_ARM</code>	<code>force:u8</code>
8193	<code>CMD_DISARM</code>	<code>force:u8</code>
8194	<code>CMD_CALIBRATE_IMU</code>	<code>which:u8</code>
8195	<code>CMD_SET_PID</code>	<code>axis:u8</code> , <code>kp:f32</code> , <code>ki:f32</code> , <code>kd:f32</code>
8196	<code>CMD_SET_GYRO_LPF</code>	<code>cutoff_hz:u16</code>
8197	<code>CMD_SET_MOTOR_GEOMETRY</code>	<code>layout:u8</code> , <code>n:u8</code> , <code>mix[]:f32</code>
8198	<code>CMD_SET_FLIGHT_MODE</code>	<code>mode:u8</code> , <code>source:u8</code>

Acked by `COMMAND_ACK` (msgid 5) carrying the command msgid + `req_seq`. New commands are added by appending a dialect entry — no envelope, no float packing.

11. Risks & Trade-offs

- **+2 header bytes & double-emit during migration** raise UART load transiently. Mitigation: dual-emit only in bring-up; the per-message rate table already exists in `telemetry_task.c` and can throttle v1 during Phase 2.
- **CRC-16 vs CRC-32.** CRC-16+`CRC_EXTRA` is the right call for small frames (better *semantic* protection than v1's bare CRC-32), but for multi-kB bulk transfers we keep an opt-in CRC-32 mode (§8.4) so error detection doesn't weaken on big blobs.
- **Codegen is upfront work.** But it deletes three hand-maintained codecs and is what permanently closes **L2/L8**. Do Phase 0 first; it pays for itself before any wire change.
- **Crypto cost & key management.** The secured trailer (`[timestamp:6][tag:8|16]`) is ~12–20 B heavier per frame than the 2-byte CRC it replaces, and AEAD costs CPU (small on the M4, but nonzero) — so mix per-frame: encrypt the command path, leave high-rate telemetry cleartext or signed-only. Key provisioning is operationally heavy; ship pre-shared keys first, ECDH later (§8.5). Never hand-roll the primitive.
- **Don't over-build.** We are not chasing MAVLink wire compatibility, just its scaling properties. Ranges, microservices, and crypto can land incrementally; the frame format (§5) + `CRC_EXTRA` (§6) + truncation (§7) are the load-bearing minimum and should ship together in Phase 1.

12. Summary Table — v1 vs v2

Concern	NavLink v1	NavLink v2
Msg-ID space	16 (4-bit)	16.7M (24-bit)
Add a message	edit 3 codecs by hand	edit dialect, regen
Version drift	silently mis-decoded	CRC fails (<code>CRC_EXTRA</code>)
Field growth	flag-day rebuild	append + truncation
Loss detection	none	per-stream <code>seq</code>
Addressing	1 byte, unused	<code>sysid</code> + <code>compid</code> + routing
Commands	fire-and-forget, 2nd ID space	typed command messages + <code>COMMAND_ACK</code> (no float box)
Params	bespoke per-setter	typed <code>PARAM_*</code> service, transactional (generation)

Concern	NavLink v1	NavLink v2
Missions	none	MISSION_* service
Bulk/logs	none (256 B cap)	FILE_TRANSFER
Security	none	toggleable AEAD: sign and/or encrypt, per-frame
Codecs	3 hand-written	1 source → generated

13. Improving on MAVLink

NavLink v2 borrows MAVLink’s *sound* mechanics (24-bit msgid, CRC_EXTRA, extension/truncation growth, seq, sysid/compid). It is worth being explicit about where it deliberately does **better** than MAVLink — these are the divergences that justify a bespoke protocol rather than just running MAVLink.

MAVLink fault	NavLink v2 solution	§
No encryption at all (signs only; confidentiality is out of scope)	Toggleable AEAD — sign and/or encrypt per frame, ChaCha20-Poly1305 default	§8.5
Clunky signing (6-byte tag, 48-bit “10 µs since 2015” counter)	AEAD tag + wall-clock-anchored 48-bit stamp (windowed + monotonic, reboot-safe, no persisted counter)	§8.5
Incoherent time (time_boot_ms rolls at 49.7 d; time_usec ambiguous; TIMESYNC awkward)	One defined [s:20\ ms:12] epoch + mandatory RTT-compensated sync (startup / 5 h / emergency)	§7.1, §8.6
7-float32 command box (can’t hold exact int32 → forced COMMAND_LONG/COMMAND_INT split; always 28 B)	Commands are typed first-class messages ; params are real typed fields; one COMMAND_ACK	§8.1, §10.7
PARAM_VALUE bit-casts ints through a float	Typed param value (real type, 8 B), sourced from the dialect	§8.2
Racy param/mission transfer (lost item undetectable; no confirm; half-applied uploads)	Generation counter + transactional / atomic-commit transfer	§8.2, §8.3
Wire field-reordering (largest-first → wire-order source-order; codegen confusion)	Declaration-order wire layout + packed structs (byte-for-byte overlay, memcpy)	§7, §9
Positional fields (order implied, evolution via convention)	Explicit per-field index , codegen-validated	§9
XML dialect sprawl + msgid squatting/collisions	Single JSON dialect, explicit ranges, JSON-Schema CI validation	§9
compat_flags byte re-sent every frame	Dropped — capabilities negotiated once at connect	§5, §10.1

What we give up (the honest cost): MAVLink’s ecosystem — QGroundControl, MAVSDK, pymavlink, ArduPilot/PX4 interop — and a decade of field-hardening. NavLink trades that for a leaner frame, native confidentiality, a coherent time base, typed commands/params, and single-source codegen. The trade only pays off because we control both endpoints and actually exploit the divergences above — not by re-skinning MAVLink.